

Documentation & Project Diary

Innovation Lab 1
Year 2025

Project: AI_CHATGPT

Team: 04

General Information

Project name: AI_CHATGPT

Supervisor: Bernhard Knapp

Innovation Lab 1 WS 2025/26

Projectteam:

Hila Jarod, if24b259@technikum-wien.at - Developer / research

Mateo Lamthi, if23b263@technikum-wien.at - Developer / benchmarking workflow

Serkan Oray, if23b243@technikum-wien.at - Developer / scoring & data

Benjamin Puka, if23b197@technikum-wien.at - Developer / UI (leaderboard)

Management Summary of the Project

The goal of AI_CHATGPT is to deliver a lightweight, reproducible benchmarking framework for large language models (LLMs). The framework manages multiple-choice benchmark sets with a review workflow (draft → in review → approved → locked), stores runs and raw model outputs in a SQLite database, scores model answers with transparent rules, and exports results as CSV/JSON plus a static HTML leaderboard that can be served locally.

Project priorities (this semester):

- Reproducible workflow for benchmark sets (import, review, approval, lock)
- Deterministic scoring and traceable storage of raw outputs and per-question scores (SQLite)
- Simple command-line interface for running evaluations and exporting results
- Human-readable results (CSV + leaderboard HTML)
- Extensible design for future API/local model adapters

Framework Conditions and Project Environment

Target users: students / researchers who want to compare several LLMs on the same benchmark in a transparent way.

System environment:

- Runs on a local laptop (Windows/macOS/Linux). No GPU required for the current file-based runner.
- Implemented in Python (standard library only) and SQLite for persistence.
- Output is generated into files (CSV/JSON/HTML) and can be viewed in any browser.

Quality and framework conditions:

- Reproducibility: every run has a run_id and stores raw model output text in the database.
- Traceability: per-question scoring is stored (answers table) plus aggregates per model (aggregates table).
- Security: API keys are not stored in the repository; future API adapters are designed to read keys from environment variables.
- Usability: clear folder conventions (imports/*, results/*) and a single CLI entry point (scripts/benchmark.py).

Important semester constraint:

A working prototype with a complete end-to-end workflow (import → prompt → collect outputs → score → export → leaderboard) is delivered in this semester. Training or fine-tuning new models and production deployment are explicitly out of scope.

Semester-Roadmap

Planned sprint roadmap and delivered milestones (effort is an estimate in person-hours).

Sprint	Time window	Main goals	Key deliverables	Est. effort (ph)
Sprint 1	Week 1–2	Project setup + requirement clarification (ORDER model)	Repository structure, backlog, first CLI skeleton	25
Sprint 2	Week 3–4	Data model + import pipeline	SQLite schema, import-questions, first benchmark set imported	35
Sprint 3	Week 5–6	Review workflow + prompt generation	submit-review, approve, lock; build-prompt to create blind test prompt files	25
Sprint 4	Week 7–8	Scoring + exports	Parser, scoring rules, run-file, export CSV/JSON, aggregates	35
Sprint 5	Week 9–10	Leaderboard + polishing	Static leaderboard (HTML+JSON), local server, examples, final cleanup	20

Collaboration & Tooling

Repository / source code:

- Git: internal / not public; the submission includes the complete source code as ZIP in Moodle.

Work management:

- ClickUp board: <https://app.clickup.com/90151777942/v/li/901516391311>

Communication:

- Weekly sprint meetings (on-site/online), quick status updates via group chat.

Development tooling:

- Python 3, SQLite, local IDE (PyCharm/VS Code).
- Basic code organisation into modules: adapters/, storage/, evaluator/, web/.

Remarks

Remarks and assumptions:

- The delivered benchmark format focuses on multiple-choice questions with answers A–E.
- Scoring rule is intentionally strict to discourage random guessing: correct = +1, blank = 0, wrong = -10.
- API-based model execution (OpenAI/Anthropic/Google) is prepared as adapter stubs; wiring requires adding the vendor SDK and handling rate limits and costs.

Brief Description of the Project

Project purpose

Large language models are often compared using public leaderboards, but reproducing results and understanding how scores were produced can be difficult. AI_CHATGPT provides a small, transparent benchmarking workflow that a student can run locally. The key idea is: a benchmark set is created once, reviewed and locked, then used to evaluate multiple models consistently. Every run stores the raw model output so scoring can be audited later.

What will be implemented (deliverables)

- Command-line benchmark tool (scripts/benchmark.py) with subcommands for the full workflow.
- SQLite database schema to store benchmark sets, questions, runs, model outputs, and scores.
- Import pipeline for benchmark questions from text files with answer keys.
- Prompt builder that produces a 'blind test' prompt file that can be pasted into any LLM.
- File-based runner that reads saved model outputs and scores them.
- Export of results as CSV and JSON plus an LM-Arena-style static leaderboard (HTML).

Greatest challenges

- Defining an unambiguous input/output format so different models can be compared fairly.
- Designing the workflow so that benchmark sets cannot be changed after models were evaluated (locking).
- Keeping the tool dependency-light while still producing usable visual output.

Scope and non-goals

In scope:

Multiple-choice benchmarks, deterministic scoring, review workflow, local leaderboard, and reproducible storage.

Out of scope (non-goals):

Training or fine-tuning models; production deployment; complex UI; adversarial evaluation beyond MCQ; automatic API calls to paid LLMs (only stubs provided).

Implementation approach

The solution is implemented as a small Python package with clear modules. The CLI orchestrates the workflow, SQLite stores all important artefacts, and the leaderboard is generated as static files to minimize dependencies.

Specification of the Solution

System environment and boundaries

Inputs:

- Benchmark question files with answer keys (imports/answer_key/*.txt).
- Optional: model output files containing answers in strict format (imports/model_outputs/*.txt).

Outputs:

- SQLite database with all artefacts (db/benchmark.db).
- Result exports (results/*): benchmark_results.csv, leaderboard.json, index.html.

Core concepts

- Question set (set_id): a collection of questions with a lifecycle status: draft → in_review → approved → locked.
- Run (run_id): one evaluation session of one set_id against one or more models.
- Model run: results for one model within one run_id (stores raw output text).

Functional requirements (features)

FR1 - Initialize database: create all required SQLite tables.

FR2 - Import benchmark set: parse .txt files, extract questions/choices and correct answers, store as a draft set.

FR3 - Review workflow: submit-review assigns a reviewer; approve confirms sign-off; lock freezes the set for use.

FR4 - Prompt generation: build-prompt writes a ready-to-copy prompt with strict answer template and all questions.

FR5 - Collect model answers: users run any LLM manually (copy/paste) and store outputs as text files.

FR6 - Parse + validate outputs: extract QID: LETTER answers and count format violations.

FR7 - Score answers: apply scoring rule mcq_v1 per question and per category; store per-question scores + aggregates.

FR8 - Export results: generate CSV + leaderboard assets (HTML + JSON) for a given run_id.

FR9 - Serve results: start a local HTTP server to view the leaderboard in a browser.

Interfaces

Command line interface (CLI):

scripts/benchmark.py with subcommands: init-db, import-questions, submit-review, approve, lock, build-prompt, run-file, export, serve.

File interfaces:

Benchmark input format: plain text files containing MCQ questions and a line 'Correct answer: X'.

Model output format: one answer per line in the form 'QID: LETTER' (blank after ':' allowed).

SQLite tables: question_sets, questions, runs, model_runs, answers, aggregates.

Data model (simplified):

Table	Purpose	Key fields
question_sets	Lifecycle management of benchmark sets	set_id, status, author, reviewer, created_at, updated_at
questions	Stores questions and correct answers for one set	(qid,set_id), category, prompt, choices_json, correct_answer
runs	One evaluation session per set	run_id, set_id, created_at, prompt_policy, notes
model_runs	Raw model outputs per run	model_run_id, run_id, model_id, source, raw_text
answers	Per-question scoring per model_run	(model_run_id,qid), given_answer, correct_answer, score
aggregates	Aggregated scores per model_run	model_run_id, total_score, per-category scores, correct/wrong/blank, format_violations

Non-functional requirements

- Portability: runs on standard Python; no external dependencies required for the core workflow.
- Maintainability: modular code structure; clear separation between storage, evaluation, adapters, and web output.
- Security: no secrets committed; future API execution uses environment variables.
- Transparency: scoring rules are explicit and results can be audited from stored raw outputs.

Architecture overview

High-level flow:

1) Import questions → 2) Review/lock → 3) Build prompt → 4) Collect outputs → 5) Parse+score → 6) Export+leaderboard

Module structure (src/):

- storage/: SQLite connection + schema
- evaluator/: parsing + scoring
- runner/: file runner (reads model outputs)
- web/: generates leaderboard HTML/JSON
- adapters/: stubs for future API model execution

Effort Estimation

Effort estimate (intuitive + backlog-based)

The ClickUp backlog was used as the basis for a rough estimate. The table below summarizes the main work packages.

Work package	Main tasks	Est. effort (ph)
Project setup	Repo structure, backlog, meeting setup	20
Benchmark data	Create/collect MCQ sets, answer keys, formatting rules	40
Database & workflow	Schema + set lifecycle (draft/review/approve/lock)	25
Prompt builder	Generate blind test prompt + template	10
Parser & scoring	Strict output parsing, scoring rules, aggregates	20
CLI integration	Subcommands, argument parsing, error handling	15
Exports	CSV/JSON output, run traceability	10
Leaderboard	Static HTML+JSON, filtering	10
Documentation	README + handover protocol + diary updates	15

Delivery

Scope of delivery

Delivered artefacts:

- Source code (Python) including CLI: scripts/benchmark.py and src/ package.
- Benchmark datasets: imports/answer_key and imports/blind_test (examples).
- Example model outputs: imports/model_outputs/*.txt.
- Example exports: results/ (CSV + leaderboard).
- SQLite database file: db/benchmark.db (can be regenerated).

Installation and setup (step-by-step)

1) Prerequisites

- Install Python 3.10+ (recommended: 3.11).
- No external Python packages are required for the current prototype (standard library only).

2) Get the project files

- Unzip the submission archive and open a terminal in the project root folder (Inno_LLM_Scoring_NextLevel/).

3) Create a virtual environment (recommended)

macOS/Linux:

```
python3 -m venv .venv
```

```
source .venv/bin/activate
Windows (PowerShell):
py -m venv .venv
.\.venv\Scripts\Activate.ps1
4) Initialize the database
python3 scripts/benchmark.py init-db --db db/benchmark.db
```

End-to-end usage protocol (example)

This example shows the complete workflow using the provided example benchmark files.

Step A -Import questions into a new set (draft)

```
python3 scripts/benchmark.py import-questions --db db/benchmark.db --source
imports/answer_key --set-id set_001 --author "Team04"
```

Step B -Two-person sign-off: submit for review and approve

```
python3 scripts/benchmark.py submit-review --db db/benchmark.db --set-id set_001 --reviewer
"ReviewerName"
```

```
python3 scripts/benchmark.py approve --db db/benchmark.db --set-id set_001 --reviewer
"ReviewerName"
```

Step C -Lock the set (freeze content)

```
python3 scripts/benchmark.py lock --db db/benchmark.db --set-id set_001
```

Step D -Build a blind-test prompt file

```
python3 scripts/benchmark.py build-prompt --db db/benchmark.db --set-id set_001 --out
imports/blind_test/set_001_prompt.txt
```

Open the generated file, copy the full prompt into an LLM chat, and ask the model to answer.

Step E -Save model outputs

Create a file for each model under imports/model_outputs/ (e.g., gpt5.txt). The file must contain only lines like:

C1: C

C2:

...

X10: B

Step F -Score a run from the saved outputs

```
python3 scripts/benchmark.py run-file --db db/benchmark.db --set-id set_001 --model-outputs
imports/model_outputs --run-id run_001 --notes "baseline comparison"
```

Step G -Export results (CSV + leaderboard)

```
python3 scripts/benchmark.py export --db db/benchmark.db --run-id run_001 --out
results/run_001
```

Step H -View the leaderboard in a browser

```
python3 scripts/benchmark.py serve --dir results/run_001/leaderboard --port 8080
```

Then open <http://localhost:8080> in a browser.

How to create your own benchmark questions

- 1) Create a new text file in imports/answer_key/ (e.g., Category_7_MyTopic.txt).
- 2) Use the structure:
QID. Question text
A. option
B. option
C. option
D. option
E. option
Correct answer: X
- 3) Keep QIDs unique and consistent (C1..C10, E1..E10, M1..M10, A1..A10, N1..N10, X1..X10).

Licenses / copyrights

- The prototype code is student work. Third-party SDKs are not included. If API adapters are implemented later, relevant vendor licenses apply.

Hardware requirements

- Standard laptop/PC. Storage requirements are small (SQLite DB + text exports).

Our Project Diary

Project Diary -Chapter 1 (Order clarification)

At the start, our team assumed the project aims to develop a framework for benchmarking large language models (LLMs) across multiple tasks and metrics, focusing on performance, alignment and usability. However, concrete requirements (metrics, supported LLMs, visualization options) were unclear. Together with our supervisor Bernhard Knapp, we clarified scope, goals and constraints. Key decisions: support at least two LLMs, provide a review workflow for benchmark sets, allow exportable reports and handle API keys securely. Scope includes a functional prototype with automated scoring; training new models and production deployment are excluded.

Project Diary -Chapter 2 (Research and specialization)

We compared existing LLM evaluation frameworks. A key observation: there is no single standard; some frameworks are domain-specific while others are broad. We also looked at security-oriented benchmarks (prompt manipulation robustness). After discussion with our supervisor, we decided to implement a compact, reproducible MCQ benchmarking workflow that can later be extended with additional tasks and adapters.

Project Diary -Chapter 3 (Architecture and first implementation)

We designed the end-to-end workflow first (import → review → prompt → score → export). To keep the prototype easy to run for anyone, we chose Python standard library only and SQLite for persistence. We implemented the database schema and a CLI skeleton with subcommands. During this phase we aligned naming conventions (set_id, run_id) and agreed on strict model output formatting to avoid ambiguous parsing.

Project Diary -Chapter 4 (Dataset and scoring rule decisions)

The team created/collected six benchmark categories (Chemistry, Emotions, Math, 3D reasoning, Admit-no-knowledge, Contradict-user-input), each with 10 MCQs. We standardized the input format and added a scoring rule that strongly penalizes wrong answers (-10) and rewards correct answers (+1), with blanks scored as 0. This was discussed as a way to measure calibrated uncertainty rather than guessing.

Project Diary -Chapter 5 (Integration, exports, and leaderboard)

After parsing and scoring were stable, we integrated result export. We generate a CSV for quick analysis and a static HTML leaderboard (with JSON data) so results can be viewed without

additional dependencies. A small local server command was added to make it one-step to open the leaderboard in a browser. We also added example model output files and validated the full pipeline with a baseline run.

Project Diary -Chapter 6 (Finalization and retrospective)

In the final sprint we focused on polishing: clearer error messages, consistent folder conventions, and documentation aimed at a non-expert reader. Main lessons learned: strict formats save debugging time; storing raw outputs is essential for traceability; and a review/lock workflow prevents accidental benchmark changes. Next steps would be wiring real API adapters (OpenAI/Anthropic/Google) and adding additional evaluation tasks beyond MCQ.